

BEGINNER LEVEL

1. Variables

variables are used to store data values like numbers or text. You don't need to declare their type; Python decides it automatically based on the value. Variable names are case-sensitive, must start with a letter or underscore, and cannot have spaces.

Ex : `a=5` # a is a variable

Code snippet:

```
[1]: #Beginner Level
# Variables
# 1.1 Create a variable named pi and store the value 22/7 in it.
pi = 22 / 7
print("Value of pi:", pi)
print("Data type of pi:", type(pi))

print("-" * 40)

# 1.2 Trying to create a variable called 'for' This will cause SyntaxError because 'for' is a reserved keyword.
for_value = 4
print("Value of for_value:", for_value)

print("-" * 40)

# 1.3 Calculate Simple Interest
P = 1000
R = 5
T = 3

SI = (P * R * T) / 100
print(f"Simple Interest for P={P}, R={R}%, T={T} years is: {SI}")

Value of pi: 3.142857142857143
Data type of pi: <class 'float'>
-----
Value of for_value: 4
-----
Simple Interest for P=1000, R=5%, T=3 years is: 150.0
```

Description:

This code finds the value and data type of π (22/7), shows that keywords like `for` can't be variable names, and calculates Simple Interest using $(P \times R \times T) / 100$.

2. Numbers

Variables are used to store data values like numbers or text. You don't need to declare their type; Python decides it automatically based on the value. Variable names are case-sensitive, must start with a letter or underscore, and cannot have spaces.

Code Snippet:

```
[2]: #2.Numbers
# 2.1. Function using format() with user input
def format_number(num, fmt):
    return format(num, fmt)

num = int(input("Enter a number to format: "))
fmt = input("Enter a format specifier : ")

result = format_number(num, fmt)
print(f"Formatted Result: {result}")
if fmt == 'o':
    print("Representation Used: Octal (Base 8)")
elif fmt == 'b':
    print("Representation Used: Binary (Base 2)")
elif fmt == 'x':
    print("Representation Used: Hexadecimal (Base 16)")
else:
    print("Representation Used: Other format")

print("-" * 40)

# 2.2 Area of Circular Pond + Water Calculation
pi = 3.14
r = float(input("Enter the radius of the pond (in meters): "))

area = pi * (r ** 2)
print("Area of the pond:", area)

# 2.3 Bonus: Water calculation
water_per_sq_meter = 1.4
water = area * water_per_sq_meter
print("Water in the pond (liters):", int(water))
```

```
# 2.3 Bonus: Water calculation
water_per_sq_meter = 1.4
water = area * water_per_sq_meter
print("Water in the pond (liters):", int(water))

print("-" * 40)

# 2.4 Calculate Speed in meters/second
distance = float(input("Enter the distance (in meters): "))
time_minutes = float(input("Enter the time taken (in minutes): "))
time_seconds = time_minutes * 60

speed = distance / time_seconds
print("Speed in m/s:", int(speed))
```

```
Enter a number to format: 2
Enter a format specifier (e.g., 'o' for octal): g
Formatted Result: 2
Representation Used: Other format
-----
Enter the radius of the pond (in meters): 145
Area of the pond: 66018.5
Water in the pond (liters): 92425
-----
Enter the distance (in meters): 120
Enter the time taken (in minutes): 7
Speed in m/s: 0
```

Description:

Format Function: Takes a number and format specifier (b, o, x) → prints number in binary, octal, or hexadecimal.

Pond Area: Takes radius → calculates area using πr^2 .

Water Calculation: Multiplies area by water per sq. meter → shows water in liters.

Speed Calculation: Takes distance & time (minutes) → converts time to seconds → calculates speed in m/s.

3.If Condition

The **if condition** is used to make decisions in a program. It checks whether a condition is True, and if it is, the code inside the if block runs. You can also use **elif** for extra conditions and **else** for what happens if none are true.

Code snippet:

```
•[8]: #3. if Con
      # BMI Calculator Program

      height = float(input("Enter height in meters: "))
      weight = float(input("Enter weight in kilograms: "))

      bmi = weight / (height ** 2)

      print(f"Your BMI is: {bmi:.2f}")

      # Determine BMI Category
      if bmi >= 30:
          print("Category: Obesity")
      elif bmi >= 25:
          print("Category: Overweight")
      elif bmi >= 18.5:
          print("Category: Normal")
      else:
          print("Category: Underweight")

      # Program to determine which country a city belongs to

      Australia = ["Sydney", "Melbourne", "Brisbane", "Perth"]
      UAE = ["Dubai", "Abu Dhabi", "Sharjah", "Ajman"]
      India = ["Mumbai", "Bangalore", "Chennai", "Delhi"]

      city = input("Enter a city name: ").strip()

      # Check which country the city belongs to
      if city in Australia:
          print(f"{city} is in Australia")
      elif city in UAE:
          print(f"{city} is in UAE")
      elif city in India:
          print(f"{city} is in India")
      else:
          print(f"Sorry, {city} is not in the list.")

      # Program to check if two cities belong to the same country

      Australia = ["Sydney", "Melbourne", "Brisbane", "Perth"]
      UAE = ["Dubai", "Abu Dhabi", "Sharjah", "Ajman"]
      India = ["Mumbai", "Bangalore", "Chennai", "Delhi"]

      city1 = input("Enter the first city: ").strip()
      city2 = input("Enter the second city: ").strip()

      def get_country(city):
          if city in Australia:
              return "Australia"
          elif city in UAE:
              return "UAE"
          elif city in India:
              return "India"
          else:
              return None

      country1 = get_country(city1)
      country2 = get_country(city2)
```

```

country1 = get_country(city1)
country2 = get_country(city2)

if country1 and country2:
    if country1 == country2:
        print(f"Both cities are in {country1}")
    else:
        print("They don't belong to the same country")
else:
    print("One or both cities are not in the list.")

```

```

Enter height in meters: 145
Enter weight in kilograms: 45
Your BMI is: 0.00
Category: Underweight
Enter a city name: sydney
Sorry, sydney is not in the list.
Enter the first city: Sydney
Enter the second city: Perth
Both cities are in Australia

```

Description:

- Calculates **BMI** from user's height and weight.
- Uses if-elif-else to categorize BMI: Underweight, Normal, Overweight, Obesity.
- Checks which **country a city belongs to** from predefined lists.
- Compares **two cities** to see if they belong to the same country.
- Demonstrates the use of **conditional statements** (if, elif, else) to control program flow.

4.Dictionary

Dictionary is a collection of **key-value pairs** used to store data. Each key is unique, and it is used to access its corresponding value. Dictionaries are **unordered** and **mutable**, meaning you can change, add, or remove items. They are defined using curly braces.

Code snippet:

```

* [7]: #4.dictionary
# 1. List of friends and their name lengths
friends = ["Ram", "Navin", "Sita", "Gaya", "Meera"]

friends_with_length = [(name, len(name)) for name in friends]
print("Friends with name lengths:")
print(friends_with_length)

your_expenses = {
    "Hotel": 1200,
    "Food": 800,
    "Transportation": 500,
    "Attractions": 300,
    "Miscellaneous": 200
}

partner_expenses = {
    "Hotel": 1000,
    "Food": 900,
    "Transportation": 600,
    "Attractions": 400,
    "Miscellaneous": 150
}

your_total = sum(your_expenses.values())
partner_total = sum(partner_expenses.values())

print("\nYour Total Expenses:", your_total)
print("Partner's Total Expenses:", partner_total)

```

```

if your_total > partner_total:
    print("\nYou spent more overall.")
elif partner_total > your_total:
    print("\nYour partner spent more overall.")
else:
    print("\nYou both spent the same amount.")

max_difference = 0
max_category = ""

for category in your_expenses:
    difference = abs(your_expenses[category] - partner_expenses[category])
    if difference > max_difference:
        max_difference = difference
        max_category = category

print(f"\nCategory with the biggest difference: {max_category}")
print(f"Difference in spending: {max_difference}")

```

```

Friends with name lengths:
[('Ram', 3), ('Navin', 5), ('Sita', 4), ('Gaya', 4), ('Meera', 5)]

```

```

Your Total Expenses: 3000
Partner's Total Expenses: 3050

```

```

Your partner spent more overall.

```

```

Category with the biggest difference: Hotel
Difference in spending: 200

```

Discription:

The program uses **dictionaries** to store and compare data. It calculates the length of friends' names, tracks expenses for you and your partner, compares total spending, and finds the category with the biggest difference. It shows how dictionaries help store, access, and analyze data efficiently.

5.For loops

A **for loop** is used to iterate over a sequence such as a list, tuple, string, or range of numbers. It executes a block of code for each item in the sequence.

Code snippet:

```

#5. for Loop
import random
print("=== Dice Rolling Simulation ===")
rolls = []
for i in range(20):
    roll = random.randint(1, 6)
    rolls.append(roll)

count_6 = rolls.count(6)
count_1 = rolls.count(1)

# Count two 6s in a row
two_sixes = 0
for i in range(len(rolls) - 1):
    if rolls[i] == 6 and rolls[i+1] == 6:
        two_sixes += 1

print("Dice Rolls:", rolls)
print("Number of times you rolled a 6:", count_6)
print("Number of times you rolled a 1:", count_1)
print("Number of times you rolled two 6s in a row:", two_sixes)

print("\n=== Jumping Jacks Workout ===")
total_jumping_jacks = 100
completed = 0

while completed < total_jumping_jacks:
    completed += 10

    if completed >= total_jumping_jacks:
        print("Congratulations! You completed the workout!")
        break

print(f"You have completed {completed} jumping jacks.")

```

```

remaining = total_jumping_jacks - completed
print(f"{remaining} jumping jacks remaining.")

tired = input("Are you tired? (yes/y or no/n): ").strip().lower()

if tired in ["yes", "y"]:
    skip = input("Do you want to skip the remaining sets? (yes/y or no/n): ").strip().lower()
    if skip in ["yes", "y"]:
        print(f"You completed a total of {completed} jumping jacks.")
        break
    else:
        continue
elif tired in ["no", "n"]:
    continue
else:
    print("Invalid input, continuing workout...")

```

```

=== Dice Rolling Simulation ===
Dice Rolls: [1, 5, 5, 1, 2, 1, 1, 4, 3, 5, 1, 1, 5, 2, 6, 5, 2, 4, 3, 4]
Number of times you rolled a 6: 1
Number of times you rolled a 1: 6
Number of times you rolled two 6s in a row: 0

```

```

=== Jumping Jacks Workout ===
You have completed 10 jumping jacks.
90 jumping jacks remaining.
Are you tired? (yes/y or no/n): no
You have completed 20 jumping jacks.
80 jumping jacks remaining.
Are you tired? (yes/y or no/n): no
You have completed 30 jumping jacks.
70 jumping jacks remaining.
Are you tired? (yes/y or no/n): yes
Do you want to skip the remaining sets? (yes/y or no/n): yes
You completed a total of 30 jumping jacks.

```

Description:

- Rolls a dice 20 times and keeps track of results.
- Counts how many times **1** and **6** appear, and checks for consecutive 6s.
- Simulates a **jumping jacks workout**, updating completed sets.
- Asks the user if they are tired or want to skip remaining sets.
- Shows how **loops** make repetitive tasks easier and handle user input.